```
$ curl -H 'Content-Type: application/json' \
  -u test:password -X POST https://localhost:4567/sessions
{"token":"eyJlbmMiOiJBMjU2R0NNIiwiYWxnIjoiZGlyIn0..hAOoOsgfGb
8yuhJD
[CA].kzhuXMMGunteKXz12aBSnqVfqtlnvvzqInLqp83zBwUW_rqWoQp5wM_q
2D7vQxpK
[CA]TaQR4Nuc-D3cPcYt7MXAJQ.ZigZZclJPDNMlP5GM1oXwQ"}
```

Compressed tokens

The encrypted JWT is a bit larger than either a simple HMAC token or the NaCl tokens from section 6.3.2. JWE supports optional compression of the JWT Claims Set before encryption, which can significantly reduce the size for complex tokens. But combining encryption and compression can lead to security weaknesses. Most encryption algorithms do not hide the length of the plaintext message that was encrypted, and compression reduces the size of a message based on its content. For example, if two parts of a message are identical then it may combine them to remove the duplication. If an attacker can influence part of a message, they may be able to guess the rest of the contents by seeing how much it compresses. The CRIME and BREACH attacks (http://breachattack.com) against TLS were able to exploit this leak of information from compression to steal session cookies from compressed HTTP pages. These kinds of attacks are not always a risk, but you should carefully consider the possibility before enabling compression. Unless you really need to save space, you should leave compression disabled.

6.4 Using types for secure API design

Imagine that you have implemented token storage using the kit of parts that you developed in this chapter, creating a `JsonTokenStore` and wrapping it in an `EncryptedTokenStore` to add authenticated encryption, providing both confidentiality and authenticity of tokens. But it would be easy for somebody to